



Machine Learning for Petroleum Engineers Using Python

Módulo 7 · Clase 4 — la última
Las Máquinas que Ven



Carlos Enrique Mosquera Trujillo



SLB Ecuador

Dónde Estamos: la Última Clase

Siete partes, y todas suman piezas para **su entrega del jueves**:

1. **La solución del reto** — Fashion-MNIST, corregido como un examen.
2. **La GPU** — el paréntesis que vale billones, y el botón que les toca.
3. **El mapa de la visión** — sesenta años de historia, la convolución, y las tareas.
4. **Cada tarea, con código** — cuatro modelos, la misma línea, y la perilla del umbral. Con el **ojo por código**: Gemini multimodal desde la celda.
5. **El oído por código** — Whisper: dictan al micrófono, y el modelo transcribe *dentro* de su Colab.
6. **Los peligros, con nombre** — la promesa pendiente, pagada.
7. **El cierre** — la puesta en común, y su entrega.

LA LLAVE DE LA CLASE

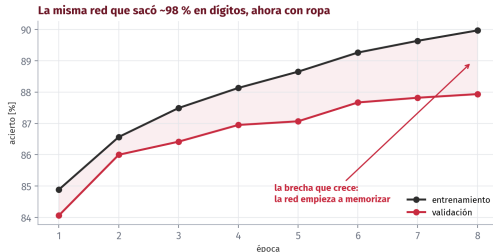
Llega por el chat del curso. Va directo a los **Secrets de Colab** (🔑), con el nombre LLAVE_CLASE. Nunca pegada en una celda — y al final de la clase la **revocamos en vivo**. Eso también es lección.

1 • La Solución del Reto

Fashion-MNIST, corregido como un examen

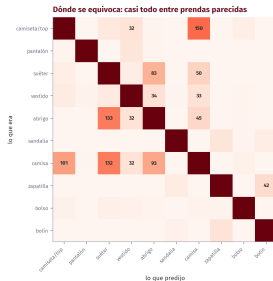
o – 18 min

La Misma Red, Otro Dato



La red que sacó **98 %** con dígitos, idéntica, da **87 %** con ropa — y sus curvas se separan: entrenamiento 90, validación 88. (Corrida de referencia; la de ustedes varía en decimales, las lecciones no.)

Dónde Se Equivoca



Casi todo el error vive entre **prendas parecidas**: camiseta que cree camisa (150 veces), abrigo que cree suéter (133), camisa que cree suéter (132). El pantalón y el bolso casi no fallan.

Los Errores, Mirados a la Cara

Los nueve errores MÁS confiados de la corrida de referencia



Los nueve errores **más confiados**. Pregunta para la sala: en 28×28 píxeles, ¿ustedes lo habrían hecho mejor?

Las Tres Lecciones de la Solución

1. **La dificultad vive en el dato, no en la arquitectura.** Misma red, mismo entrenamiento: 98 contra 87.
2. **Las confusiones son las humanas.** Camiseta↔camisa, abrigo↔suéter — el modelo falla donde fallaríamos nosotros.
3. **El techo no se sube con más épocas** — eso solo memoriza, y lo dicen sus propias curvas.

El techo se sube cambiando de arquitectura: una que entienda que un píxel y sus vecinos forman bordes y texturas. Esa idea se llama convolución — y es la puerta de todo lo que sigue hoy.

Nuestra red Dense miraba 784 píxeles sueltos, sin saber cuáles eran vecinos. Una red convolucional aprende patrones locales y los compone. Es la misma idea de apilar dobleces — llevada al espacio.

2 • El Paréntesis que Vale Billones

Qué es una GPU, y por qué hay que encenderla

18 – 26 min

CPU y GPU, Desde Cero

¿Notaron que entrenar la red **tardó**? Corrió en la CPU. La diferencia, sin jerga:

CPU — el cirujano

Unos pocos núcleos, rapidísimos y versátiles: hacen *cualquier* cosa, una tras otra. Perfecta para lógica, decisiones, tareas distintas encadenadas.

GPU — la cuadrilla

Miles de núcleos pequeños que hacen *la misma operación sobre muchos datos a la vez*. Mil obreros, el mismo movimiento, al unísono.

Una red neuronal es, por dentro, casi pura multiplicación de matrices: millones de operaciones idénticas e independientes. Exactamente el trabajo de la cuadrilla.

De los Videojuegos a 4 Billones de Dólares

- 1993** Nace **NVIDIA** para una sola cosa: dibujar videojuegos — millones de píxeles en paralelo. La matemática correcta, sin que nadie supiera aún para qué más serviría.
- 2007** **CUDA**: NVIDIA abre sus tarjetas al cómputo general. Los científicos descubren la cuadrilla.
- 2012** El momento bisagra: **AlexNet**, una red entrenada en **dos GPUs de videojuegos**, arrasa la competencia de reconocimiento de imágenes ImageNet. Arranca la era del deep learning.
- 2026** NVIDIA es de las empresas más valiosas del mundo — del orden de los **4 billones de dólares** (billones *en español*: millones de millones — lo que la prensa en inglés llama *trillions*) — y la GPU es el recurso estratégico de la IA: se compra por decenas de miles, con listas de espera y geopolítica de por medio.

Y el botón que a ustedes les toca: Colab → Entorno de ejecución → Cambiar tipo → GPU (T4) — gratis.

Ojo: al cambiar, **la sesión se reinicia** — se vuelven a correr las celdas. Háganlo ahora, antes de la parte de visión.

3 • El Mapa de la Visión

Cada tarea es una pregunta distinta

26 – 40 min

Sesenta Años de Humildad, en una Lámina

1966 El MIT asigna «resolver la visión por computadora» como **proyecto de verano** para estudiantes. En serio.

1970–2010 Décadas de **reglas escritas a mano**: detectores de bordes, esquinas, plantillas. Funcionan en el laboratorio; se rompen en la calle.

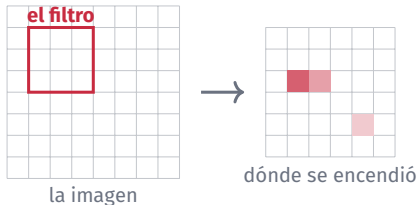
2012 AlexNet — la historia de la GPU, vista desde el otro lado: dejar de programar reglas y **aprender de ejemplos**. La visión «se resuelve», 46 años después de aquel verano.

Todo lo que van a correr hoy existe porque esa apuesta ganó: ejemplos, no reglas.

La misma idea de todo el curso — el modelo aprende del dato — solo que aquí el dato es la imagen.

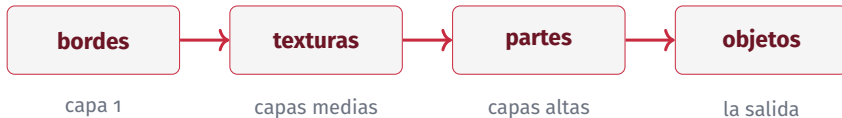
La Puerta Prometida: la Convolución

El MLP **aplana** la imagen y pierde a los vecinos. La convolución hace lo contrario: un **filtro pequeño** (3×3) que **barre** la imagen buscando *un patrón*.



Un filtro busca un borde; otro, una esquina. Y la red **no los trae de fábrica: los aprende del dato**, igual que el MLP sus pesos.

Capa Sobre Capa: la Jerarquía



Las primeras capas aprenden bordes; las siguientes los combinan en texturas; luego en partes (una rueda, una manga); al final, en objetos (un bus, una camisa). **Nadie programó esa escalera: emerge del entrenamiento.**

Por eso el techo de Fashion-MNIST se sube con convolución y no con más épocas: la diferencia entre camiseta y camisa es textura y forma — cosas de vecinos, no de píxeles sueltos.

Las Tareas, y la Pregunta que Responde Cada Una

Tarea	La pregunta	En su mundo
Clasificación	<i>¿qué es esta imagen?</i>	¿junta corroída o sana?
Detección	<i>¿qué hay, y dónde?</i>	cascos y chalecos en cámara
Segmentación	<i>¿qué píxeles exactamente?</i>	el área exacta del derrame
Pose	<i>¿en qué posición?</i>	ergonomía; persona caída
OCR / lectura	<i>¿qué dice?</i>	medidores, placas, documentos
Multimodal	<i>cualquier pregunta</i>	«describe esta foto»

Saber cuál pedir es el 80 % del trabajo — es la versión visual de «especificar bien».

Las cuatro primeras las corremos ahora, **con código, una por una**. Las dos últimas, con el ojo por código al final de la parte que sigue — y luego un sentido que no está en la tabla: **el oído**.

4 · Cada Tarea, con Código

Cuatro modelos, la misma línea — y el ojo por código

40 – 90 min

La Lámina Completa: una Foto, Cuatro Preguntas

La misma foto, cuatro preguntas — cuatro modelos, la misma línea de código

CLASIFICACIÓN · ¿qué es? — «minibus» 57 % (y duda)



DETECCIÓN · ¿qué y dónde? — 1 bus, 4 personas



SEGMENTACIÓN · ¿qué píxeles? — máscaras exactas



POSE · ¿en qué posición? — 17 puntos por persona



Corrida real, no ilustración. Los cuatro modelos de **Ultralytics** (yolo11n), cada uno con **la misma línea de código**. Vamos una por una.

4.1 · Clasificación — y una Lección Gratis

```
r = YOLO("yolo11n-cls.pt")("foto.jpg")[0]

for i in r.probs.top5[:3]:          # las 3 mejores apuestas
    print(r.names[i], float(r.probs.data[i]))
# minibus 0.57 | police_van 0.34 | trolleybus 0.04
```

El resultado se **lee**, no solo se mira: `r.probs` trae la probabilidad de cada clase.

FÍJENSE: EL MODELO DUDA

minibus 57 %, police_van 34 %. Una sola etiqueta para toda la imagen es poco cuando la imagen tiene muchas cosas — por eso existe la siguiente tarea.

4.2 · Detección — el Conteo Sale de las Cajas

```
r = YOLO("yolo11n.pt")("foto.jpg")[0]

conteo = Counter(r.names[int(c)] for c in r.boxes.cls)
# {'bus': 1, 'person': 4}   confianzas: 0.94, 0.89, 0.88, ...
```

`r.boxes`: una caja por objeto — clase, confianza y coordenadas. Y la frase que convierte la demo en herramienta:

El conteo sale de `r.boxes`, no del dibujo. Una app de EPP filtra esas cajas («persona sin casco») — no mira la imagen bonita.

4.3 · Segmentación — la Caja Dice Dónde; la Máscara, Cuánto

```
r = YOLO("yolo11n-seg.pt")("foto.jpg")[0]

r.masks.data.shape          # (6, 640, 480): objetos x alto x ancho
int(r.masks.data[0].sum())   # pixeles exactos del primer objeto
```

La máscara permite **medir**: área de la mancha, fracción de la imagen, crecimiento entre dos fotos de inspección.

Detalle de la corrida real: la segmentación encontró además un stop sign que la detección no reportó — modelos distintos, sensibilidades distintas. Verificar, siempre.

4.4 • Pose — Reglas de Seguridad Sobre Geometría

```
r = YOLO("yolo11n-pose.pt")("foto.jpg")[0]

len(r.keypoints)           # 4 personas
r.keypoints.data.shape[1:] # (17, 3): articulaciones x (x, y, conf)
```

17 puntos del cuerpo por persona. Con eso se responde «¿está agachado?», «¿levantó los brazos?», «¿hay alguien en el suelo?» — reglas de seguridad escritas sobre coordenadas.

Ahora con SUS fotos: suban una (panel ) , cambien "foto . jpg" por la suya, y anoten un acierto y un fallo.

La regla de siempre: nada de instalaciones, documentos ni personas sin permiso.

4.5 · La Perilla que Hay que Conocer: el Umbral

```
for umbral in [0.6, 0.25, 0.1]:  
    r = modelo("foto.jpg", conf=umbral, verbose=False)[0]  
    print(umbral, collections.Counter(r.names[int(c)] for c in r.bboxes.cls))
```

Umbral	Lo que reporta (corrida real)
0.6	1 bus + 4 personas
0.25	idéntico — la detección más tímida está al 62 %
0.1	...y un monopatín al 12 % que no existe

Umbral alto = se pierden objetos reales. **Umbral bajo** = entran fantasmas. En una app de seguridad esa perilla es una **decisión de negocio**: ¿qué cuesta más — la alarma falsa, o el casco sin detectar?

4.6 • ¿De Dónde Salen las 80 Clases?

El detector conoce 80 objetos porque se entrenó con **COCO**: unas 330.000 fotos cotidianas etiquetadas a mano. Por eso sabe qué es un *frisbee* y no sabe qué es una *válvula*.

EL SESGO DE CATÁLOGO

El modelo solo conoce lo que su dataset le enseñó. Es la lección de todo el curso — **el modelo hereda a su dato** — vestida de visión.

¿Y un detector de «brida corroída» o «casco de la empresa»? Para eso existe el **ajuste fino** — siguiente lámina.

4.7 · ¿Y si No Conoce SU Objeto? El Ajuste Fino

Desde cero costaría millones. En cambio se toma el modelo preentrenado y se le **re-enseña el final** con fotos propias:

1. **50-200 fotos** propias del objeto, variadas.
2. **Etiquetarlas:** cajas dibujadas a mano (hay herramientas gratuitas para eso).
3. Y una línea:

```
modelo = YOLO("yolo11n.pt")  
modelo.train(data="mis_fotos.yaml", epochs=50)
```

No lo corremos hoy — pero es lo primero que van a necesitar cuando esto toque su trabajo real, y ya saben pedirlo.

Se llama *transfer learning*: el mismo truco que hace posible todo lo de esta clase.

El Ojo por Código: la Pregunta Libre

Las cuatro tareas tienen **catálogos fijos** (80 clases, 17 puntos). Para la pregunta libre — «¿cuántos llevan casco?», «¿qué marca este medidor?» — se llama a un modelo **multimodal** por código.

LA LLAVE DE LA CLASE, EN TRES PASOS

- 1 · Cópienla del chat del curso.
- 2 · Panel  de Colab → LLAVE_CLASE, con acceso del cuaderno.
- 3 · **Nunca** en una celda.

Al final de la clase la revocamos en vivo: una llave compartida es una llave quemada. Eso es gestión de secretos.

Plan B si la llave no corre en su cuenta: el panel de Gemini acepta imágenes — misma pregunta, sin código. Pero la versión por código es la que se integra a una app.

Tres Preguntas Sobre la Misma Foto

```
from google.colab import userdata
from google import genai
from PIL import Image

cliente = genai.Client(api_key=userdata.get("LLAVE_CLASE"))
foto = Image.open("foto.jpg")

r = cliente.models.generate_content(
    model="gemini-2.5-flash",
    contents=[foto, "¿Cuántas personas se ven? Solo el número."])
print(r.text)
```

Tres pedidos en el cuaderno: **describir**, **contar** y **leer** (ahí vive el OCR).

La advertencia al cubo: cada foto enviada por esta vía **sale hacia un tercero**. Con la de la planta — parte 6, en un momento.

5 · El Oído por Código

Whisper: dictar, transcribir, estructurar

90 – 102 min

La Visión No Es el Único Sentido Preentrenado

Whisper es el modelo de voz que OpenAI liberó como **código abierto** en 2022: 680.000 horas de audio, español y 98 idiomas más, un `pip install`. Dicta usted; escribe él.

LA DIFERENCIA QUE IMPORTA

Gemini: la foto **sale** hacia un tercero. Whisper: corre **dentro de su Colab** — el modelo se descarga una vez y el audio no viaja a ninguna parte. La misma tarea, dos arquitecturas de privacidad opuestas. **Elegir entre ellas también es ingeniería.**

El plan: dictan un **reporte de campo** al micrófono → Whisper lo vuelve texto (local) → Gemini vuelve ese texto un **acta estructurada**. Voz → texto → estructura, en tres celdas.

Dictar y Transcribir

```
AUDIO = grabar(15)    # el micrófono del navegador, desde la celda

import whisper
modelo_voz = whisper.load_model("base")          # 74 MB, corre local
print(modelo_voz.transcribe(AUDIO, language="es")["text"])
```

La función `grabar()` viene en el cuaderno (un poco de JavaScript del navegador; pide permiso de micrófono la primera vez).

DICTEN COMO SI LLAMARAN DESDE CAMPO

«Inspección del tanque tres. Se observa corrosión en la brida norte. Presión registrada: mil doscientos cincuenta psi. Se recomienda cambio de empaque antes del viernes.»

De la Voz al Acta: el Flujo Completo

```
r = cliente.models.generate_content(  
    model="gemini-2.5-flash",  
    contents=f"""Reporte de campo dictado por voz: "{TEXTO}"  
    Conviértelo en un acta breve:  
    - RESUMEN: (una frase)  
    - DATOS: (equipos, valores y unidades mencionados)  
    - ACCIONES: (qué se pide hacer, con plazo si lo hay)"""  
    print(r.text)
```

Quince segundos de voz → un acta con los datos y las acciones separados, lista para un correo o un sistema. Hace tres años, esto era un producto que se compraba.

La frontera de privacidad, cruzada dos veces: el *audio* nunca salió de su Colab (Whisper es local); el *texto* sí viajó a Gemini. Este diseño les deja decidir **qué cruza y qué no** — de eso, la parte que sigue.

6 • Los Peligros, con Nombre

La promesa pendiente, pagada

102 – 115 min

6.1 • Lo que Se Pega, Sale de la Empresa

Los proveedores de estos servicios pueden **retener** lo que se les envía — prompts, datos, fotos — por meses, y **personas** pueden revisarlo. No es teoría: está en la letra chica de casi todos, incluido el Gemini de Colab (retención de hasta 18 meses, revisores humanos).

- 🔒 El dato confidencial **no se pega**: producción real, presiones, coordenadas, nombres de pozos o clientes, fotos de instalaciones.
- ✓ El **esquema sí**: nombres de columnas y tipos, sin valores. El agente escribe el código; el código corre localmente sobre el dato real.
- ? En la duda: seguridad de la información, **antes** — no después.

6.2 • Las Llaves Son Contraseñas

Lo acaban de vivir: la llave fue al **Secret**, no a la celda. ¿Por qué tanto cuidado?

- Un cuaderno **se comparte**, se sube a un repositorio, se proyecta en una reunión. Una llave pegada en una celda queda expuesta en todos esos lugares a la vez.
- Una llave expuesta es **plata**: alguien consume el servicio a nombre de ustedes — o algo peor con lo que la llave alcance.
- Por eso: secrets, **rotación**, y revocar cuando se compartió de más.

La nuestra muere hoy, delante de ustedes. Una llave compartida es una llave quemada — y quemarla a tiempo es el trabajo bien hecho.

6.3 • Las Librerías Alucinadas

Un agente puede recomendar, con total confianza, una librería **que no existe**. Y hay algo peor: que *sí* exista — porque alguien la registró con el nombre que los agentes suelen inventar, cargada de código malicioso (*typosquatting* y *slopsquatting*: registrar el error esperable).

LA DEFENSA: 30 SEGUNDOS ANTES DE CADA pip install

- ▶ ¿Existe en PyPI, con página y documentación reales?
- ▶ ¿Cuántas descargas, desde cuándo, quién la mantiene?
- ▶ ¿De verdad hace falta? Si el agente propone una librería exótica para algo que pandas ya hace — desconfíen.

Las de este curso — pandas, scikit-learn, keras, gradio, ultralytics, shap — son masivas y auditadas. La regla es para la que no conocen.

6.4 · El Peor de Todos: Código que Corre y Está Mal

No lanza error. Produce un número. El número es basura. Lo vieron **tres veces** en este módulo:

- ✗ Las propinas en efectivo: **cero** que era ausencia (C1)
- ✗ La columna a live: accuracy **1.000** que era trampa (C3)
- ✗ El tasador: **USD 397** por una piedra de 0 mm (C2)

LA DEFENSA ES LA DE LA CLASE 1 — Y NO CADUCA

- 1 · ¿Cuántas filas entraron y cuántas salieron?
- 2 · ¿Ceros, vacíos o números perfectos?
- 3 · ¿Tiene sentido físico o de negocio?
- 4 · ¿Qué decisión se toma con esto, y qué pasa si está mal?

Cuatro preguntas, dos minutos, cero programación. Es la herramienta más barata y más rentable de todo el diplomado.

7 • Cierre

La puesta en común, y su entrega

115 – 120 min

La Puesta en Común

Antes de cerrar el módulo, tres preguntas para la sala — una respuesta por equipo, en voz alta:

1. ¿Qué proceso de su trabajo **automatizarían el lunes** con lo visto en el módulo?
2. ¿Cuál fue el **error del agente** que cazaron esta semana — y cómo lo detectaron?
3. ¿Qué decisión **no le delegarían jamás** a un modelo, por buena que sea su métrica?

La tercera es la que importa: saber qué NO delegar es la forma más madura de saber usar la herramienta.

El Instructivo, 1 de 2: Ustedes Escogen el Proyecto

Equipos de hasta 3. Una tarea **afín a su área** — no necesariamente laboral — con un **dataset relacionado**. Vale cualquier tipo de dato:

-  **Tabular** — predicción o clasificación, al estilo Titanic o diamantes.
-  **Visión** — al estilo Fashion-MNIST, o detección sobre fotos propias.
-  Lo que su área pida: mantenimiento, transporte, calidad, administración, medio ambiente...

El dataset: **público** — seaborn, keras .datasets, UCI, Kaggle, datos abiertos. **Nunca datos confidenciales de la empresa.**

El Instructivo, 2 de 2: UN Cuaderno que Corre Completo

Se entrega un **.ipynb** que corre de arriba a abajo, con:

- ✓ **EDA** — las preguntas al dato antes de modelar, con gráficas que digan algo
- ✓ **Limpieza** — con las decisiones **escritas**: qué se botó, qué se imputó, por qué
- ✓ **Varios modelos con grilla** — GridSearchCV, 2-3 modelos e hiperparámetros, validación cruzada
- ✓ **Examen reservado** — el conjunto de prueba se toca **una** vez, al final
- ✓ **Matriz de costos** — qué error cuesta más y qué umbral eligieron, en el idioma del negocio
- ✓ **Explicabilidad** — SHAP o equivalente, global y por caso
- ✓ Los **prompts** usados, y **un error del agente cazado** — con cómo lo detectaron

La **app en Gradio** con su link: *opcional* — suma, pero el cuaderno es la entrega. La plantilla es el **Titanic de la Clase 3**, de la primera celda a la última. Y el error cazado vale tanto como el modelo: demuestra que dirigieron, no que obedecieron.

El Módulo, en Cuatro Clases

Clase	Lo que quedó
1	Dirigir un agente: los cuatro fundamentos, y verificar siempre
2	Un modelo servido: certificar, entrenar, examinar, blindar
3	El flujo profesional (GridSearch, SHAP) — y su primera red
4	Las máquinas que ven — y oyen: visión, voz, y los peligros

El agente escribe. Ustedes responden.

Escribir se volvió barato. El criterio para especificar bien, verificar siempre y desconfiar de lo perfecto — eso es lo que ustedes ponen, y vale más ahora que hace tres años.

Datos y Referencias

-  **Fashion-MNIST:** Xiao, Rasul & Vollgraf (Zalando Research, 2017), vía `keras.datasets`. **Foto de prueba:** assets de Ultralytics.
-  **YOLO:** Redmon et al. (2016); implementación **Ultralytics** (yolo11n: cls/det/seg/pose). **AlexNet:** Krizhevsky, Sutskever & Hinton (2012). **Gemini API:** SDK `google-generativeai`. **Whisper:** Radford et al. (OpenAI, 2022), código abierto. **COCO:** Lin et al. (2014).
-  La solución de referencia y las imágenes anotadas salen de `figuras.py` (semilla fija; corridas reales, no ilustraciones). La solución en vivo se corre con Keras y varía en decimales.

Como siempre: corren los scripts y tiene que dar lo mismo. Si no da, es un error nuestro y queremos saberlo.

¡Gracias!

Carlos Enrique Mosquera Trujillo

✉ cmosquerat@unal.edu.co

Machine Learning for Petroleum Engineers Using Python

Módulo 7 · Clase 4 · SLB Ecuador · UDLA · 2026